

CHAPTER 10 = Creating Text Embedding Models

```
# Install or upgrade the required libraries to the specified versions
!pip install -U \

    # used for creating and training embedding models
    "sentence-transformers==3.4.1" \

    # provides pretrained Transformer models like BERT
    "transformers==4.46.3" \

    # used to download and manage pretrained models
    "huggingface-hub==0.26.2" \

    # used to load datasets such as MNLI and STSB
    "datasets==3.1.0"
```



```

Collecting sentence-transformers==3.4.1
  Downloading sentence_transformers-3.4.1-py3-none-any.whl.metadata (10 kB)
Collecting transformers==4.46.3
  Downloading transformers-4.46.3-py3-none-any.whl.metadata (44 kB)
  44.1/44.1 kB 2.8 MB/s eta 0:00:00
Collecting huggingface-hub==0.26.2
  Downloading huggingface_hub-0.26.2-py3-none-any.whl.metadata (13 kB)
Collecting datasets==3.1.0
  Downloading datasets-3.1.0-py3-none-any.whl.metadata (20 kB)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from sentence-tr
Requirement already satisfied: torch>=1.11.0 in /usr/local/lib/python3.12/dist-packages (from se
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (from ser
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from sentence-t
Requirement already satisfied: Pillow in /usr/local/lib/python3.12/dist-packages (from sentence-
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from transfo
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from trar
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from trar
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from transfo
Collecting tokenizers<0.21,>=0.20 (from transformers==4.46.3)
  Downloading tokenizers-0.20.3-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metac
Requirement already satisfied: safetensors>=0.4.1 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: dill<0.3.9,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from datasets=
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from datasets=
Requirement already satisfied: multiprocess<0.70.17 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: aiohttp in /usr/local/lib/python3.12/dist-packages (from datasets
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from ai
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (f
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from rec
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: setuptools<82 in /usr/local/lib/python3.12/dist-packages (from to
Requirement already satisfied: svmov>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from to

```

Generating Contrastive Examples

```

Requirement already satisfied: cuda-toolkit==12.8.1 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: cuda-bindings<12.9.0, >=12.8.0 in /usr/local/lib/python3.12/dist-pack

```

Load and Prepare the MNLI Training Dataset

```

Requirement already satisfied: cuda-toolkit==12.8.1 in /usr/local/lib/python3.12/dist-pack

```

```
# from the dataset library import load_dataset function
from datasets import load_dataset

train_dataset = load_dataset(

    # it has collection of different nlp datasets
    "glue",

    # load the mnli dataset from GLUE collection
    "mnli",

    # give the training portion of dataset
    split="train"

).select(range(50_000)) # Selecting 50k rows

# it has 4 cols (premise , hypothesis , label , idx)
# remove idx column (no need for training)
train_dataset = train_dataset.remove_columns("idx")
```

Found existing installation: transformers 3.15.1
 /usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
 The secretsHF_TOKEN is not installed, existing secrets.
 To interact with all the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings/tokens>).
 You will be able to use this token in all of your notebooks.
 Please note that authentication is recommended but still optional to access public models or datasets.
 warn once you fully uninstall datasets-2.18.0

Generating train split: 392702/392702 [00:00<00:00, 719463.32 examples/s]
 Found existing installation: sentence-transformers 5.6.0
 Generating training sentence transformers: 9815/9815 [00:00<00:00, 262000.52 examples/s]
 Successfully installed sentence-transformers-5.6.0
 Generating validation split: 9832/9832 [00:00<00:00, 282997.51 examples/s]
 Error: Invalid value for parameter 'split': 100% : 9832/9832 [00:00<00:00, 282997.51 examples/s]
 diffusers 0.39.0 requires huggingface-hub<2.0.0, but you have huggingface-hub 0.26.2 which
 grade 0.20.0 requires huggingface-hub<2.0.0, but you have huggingface-hub 0.26.2 which is
 Generating test split: 9796/9796 [00:00<00:00, 287445.10 examples/s]
 Successfully installed datasets-3.1.0 huggingface-hub-0.26.2 sentence-transformers-5.6.0
 Generating test mismatched split: 100% : 9847/9847 [00:00<00:00, 292609.97 examples/s]
 WARNING: The following packages were not installed in this runtime:
 [datasets,huggingface_hub]
 You must restart the runtime in order to use newly installed versions.

```
# it shows the relation between the 2 sentences
train_dataset[49999]
```

```
{'premise': "Democrats think they're immune to this attack because they've got both ends of the  
spectrum On the removal question, the polls are on Clinton's side, and on the moral question,  
on which the polls are against Clinton, Democrats have acknowledged and condemned his  
misconduct.",  
'hypothesis': 'Democrats think they have covered both ends of the question.',  
'label': 0}
```

Create the Base Embedding Model

```
# import Sentence transformer to create meaning vector representation for sentences
from sentence_transformers import SentenceTransformer

# Use a base model (bert base)
# create a BERT-based Sentence Transformer model.
embedding_model = SentenceTransformer('bert-base-uncased')
```

WARNING:sentence_transformers.SentenceTransformer:No sentence-transformers model found with name

Configure the Softmax Loss Function

```
# imports the losses module from Sentence Transformers
# loss function tells the model whether the prediction was correct or wrong
from sentence_transformers import losses
```

```
# Define the loss function. In softmax loss, we will also need to explicitly set the number of
train_loss = losses.SoftmaxLoss(
    model=embedding_model,
    sentence_embedding_dimension=embedding_model.get_sentence_embedding_dimension(),
    num_labels=3
)
```

Create the STSB Evaluation Dataset

```
# Import the evaluator used to evaluate the embedding model
from sentence_transformers.evaluation import EmbeddingSimilarityEvaluator

# Load the validation split of the STSB (Semantic Textual Similarity Benchmark) dataset
val_sts = load_dataset("glue", "stsb", split="validation")

# Create an evaluator for measuring the model's performance on semantic similarity
evaluator = EmbeddingSimilarityEvaluator(

    # First sentence in each sentence pair
    sentences1=val_sts["sentence1"],

    # Second sentence in each sentence pair
    sentences2=val_sts["sentence2"],

    # Normalize similarity scores from the range 0-5 to 0-1
    scores=[score / 5 for score in val_sts["label"]],

    # Use cosine similarity to compare the sentence embeddings
    main_similarity="cosine",
)
```

```
train-00000-of-502k/502k [00:00<00:00, 900kB/s]
00001.parquet: 100%

validation-00000-of-151k/151k [00:00<00:00, 457kB/s]
00001.parquet: 100%

test-00000-of-00001.parquet: 100% 114k/114k [00:00<00:00, 347kB/s]

Generating train split: 100% 5749/5749 [00:00<00:00, 206874.23 examples/s]
```

Configure the Training Settings

```
# Import the class used to define the training configuration
from sentence_transformers.training_args import SentenceTransformerTrainingArguments

# Define the training arguments for the embedding model
args = SentenceTransformerTrainingArguments(

    # Folder where the trained model and checkpoints will be saved
    output_dir="base_embedding_model",

    # Train the model for 1 complete pass through the training dataset
    num_train_epochs=1,

    # Number of training examples processed in one batch on each device (GPU/CPU)
    per_device_train_batch_size=32,

    # Number of evaluation examples processed in one batch on each device
    per_device_eval_batch_size=32,
```

```
# Number of warmup steps before reaching the full learning rate
warmup_steps=100,

# Enable mixed precision (FP16) training to reduce memory usage and speed up training
fp16=True,

# Evaluate the model after every 100 training steps
eval_steps=100,

# Display the training loss after every 100 training steps
logging_steps=100,
)
```

```
# Train the Embedding Model with Softmax Loss
```

```
# Import the trainer class used to train the embedding model
from sentence_transformers.trainer import SentenceTransformerTrainer

# Create the trainer by providing all the required components
trainer = SentenceTransformerTrainer(

    # The embedding model to be trained
    model=embedding_model,

    # The training configuration (epochs, batch size, etc.)
    args=args,

    # The dataset used for training the model
    train_dataset=train_dataset,

    # The loss function used to calculate the training loss
    loss=train_loss,

    # The evaluator used to measure the model's performance during training
    evaluator=evaluator,
)

# Start the training process
trainer.train()
```

```
wandb: WARNING The `run_name` is currently set to the same value as `TrainingArguments.output_dir`
/usr/local/lib/python3.12/dist-packages/notebook/notebookapp.py:191: SyntaxWarning: invalid escape
  | | | | ' _ \ _ ' / _ ' | _ / -_)
wandb: (1) Create a W&B account
wandb: (2) Use an existing W&B account
wandb: (3) Don't visualize my results
wandb: Enter your choice: 3
wandb: You chose "Don't visualize my results"
wandb: Using W&B in offline mode.
wandb: W&B API key is configured. Use `wandb login --relogin` to force relogin
Tracking run with wandb version 0.28.0
W&B syncing is set to `offline` in this directory. Run `wandb online` or set WANDB_MODE=online to
enable cloud syncing.
Run data is saved locally in /content/wandb/offline-run-20260728_081157-bix9af2y
View this run in the terminal with `wandb leet`
wandb: Detected [huggingface_hub.inference] in use.
wandb: Use W&B Weave for improved LLM call tracing. Install Weave with `pip install weave` then
wandb: For more information, check out the docs at: https://weave-docs.wandb.ai
WARNING:sentence_transformers.data_collator:Column 'hypothesis' is at index 1, whereas a column
dataset = dataset.select_columns(['hypothesis', 'entailment', 'contradiction'])
[1563/1563 05:43, Epoch 1/1]
```

Step Training Loss

100	1.066100
200	0.931200
300	0.877700
400	0.835000
500	0.819900
600	0.818800
700	0.799500
800	0.786200
900	0.770100
1000	0.764000
1100	0.745400
1200	0.727800
1300	0.744800
1400	0.712100
1500	0.744400

```
TrainOutput(global_step=1563, training_loss=0.8063716241662043, metrics={'train_runtime':
689.8739, 'train_samples_per_second': 72.477, 'train_steps_per_second': 2.266, 'total_flos':
0.0, 'train_loss': 0.8063716241662043, 'epoch': 1.0})
```

```
# Evaluate our trained model
evaluator(embedding_model)
```

```
{'pearson_cosine': np.float64(0.583962084960906),
'spearman_cosine': np.float64(0.654501507420397)}
```

MTEB

```
# Install the MTEB library used to evaluate embedding models
!pip install -U mteb
```

Requirement already satisfied: pydantic-core==2.46.4 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from sentence-transformers)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from sentence-transformers)
Requirement already satisfied: Pillow in /usr/local/lib/python3.12/dist-packages (from sentence-transformers)
Requirement already satisfied: setuptools<82 in /usr/local/lib/python3.12/dist-packages (from sentence-transformers)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from sentence-transformers)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from sentence-transformers)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0)
Requirement already satisfied: cuda-toolkit==12.8.1 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0)
Requirement already satisfied: cuda-bindings<13,>=12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0)
Requirement already satisfied: nvidia-cudnn-cu12==9.19.0.56 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0)
Requirement already satisfied: nvidia-cusparse-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0)
Requirement already satisfied: nvidia-nccl-cu12==2.28.9 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0)
Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0)
Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0)
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1.* in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90.* in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0)
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83.* in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0)
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3.* in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90.* in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0)
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90.* in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0)
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90.* in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0)
Requirement already satisfied: nvidia-cusparselt-cu12==12.5.8.93.* in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93.* in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93.* in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0)
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90.* in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers)

Requirement already satisfied: tokenizers<0.21,>=0.20 in /usr/local/lib/python3.12/dist-packages (from transformers)

Requirement already satisfied: safetensors>=0.4.1 in /usr/local/lib/python3.12/dist-packages (from transformers)

Requirement already satisfied: cuda-pathfinder~=1.1 in /usr/local/lib/python3.12/dist-packages (from sentence-transformers)

Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp)

Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp)

Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp)

Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from aiohttp)

Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from aiohttp)

Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp)

Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp)

Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown)

Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy)

Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2)

Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas)

Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas)

Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas)

Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from pandas)

Downloading mteb-2.18.7-py3-none-any.whl (6.0 MB)
6.0/6.0 MB 118.1 MB/s eta 0:00:00
Downloading pytreceval-terrier-0.5.10-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (304.8 kB)
304.8/304.8 kB 30.8 MB/s eta 0:00:00
Installing collected packages: pytreceval-terrier, mteb
Successfully installed mteb-2.18.7 pytreceval-terrier-0.5.10

```
# Install MTEB version below 2.0.0 for compatibility with the current Sentence-Transformers version
!pip install "mteb<2.0.0"
```



```
Requirement already satisfied: dill<0.3.9,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from dataset
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from dataset
Requirement already satisfied: multiprocessing<0.70.17 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: fsspec<=2024.9.0,>=2023.1.0 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: aiohttp in /usr/local/lib/python3.12/dist-packages (from datase
Requirement already satisfied: huggingface-hub>=0.23.0 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from data
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from da
Requirement already satisfied: polars-runtime-32==1.35.2 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: pydantic-core==2.46.4 in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from r
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: transformers<5.0.0,>=4.41.0 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: Pillow in /usr/local/lib/python3.12/dist-packages (from sentenc
Requirement already satisfied: setuptools<82 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch>1
Requirement already satisfied: cuda-toolkit==12.8.1 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: cuda-bindings<13,>=12.9.4 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: nvidia-cudnn-cu12==9.19.0.56 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist
Requirement already satisfied: nvidia-nccl-cu12==2.28.9 in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1.* in /usr/local/lib/python3.12/dis
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90.* in /usr/local/lib/python3.1
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83.* in /usr/local/lib/python3.12/dis
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3.* in /usr/local/lib/python3.12/dis
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90.* in /usr/local/lib/python3.12/
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90.* in /usr/local/lib/python3.12/di
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90.* in /usr/local/lib/python3.12/
Requirement already satisfied: nvidia-cusparse-cu12==12.5.8.93.* in /usr/local/lib/python3.12/
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93.* in /usr/local/lib/python3.12/d
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93.* in /usr/local/lib/python3.12/
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90.* in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: cuda-pathfinder~=1.1 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (f
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages
```

```
# Load the Base Embedding Model
```

```
# Import the SentenceTransformer class used to load or create embedding models
from sentence_transformers import SentenceTransformer
```

```
# Load the base BERT model and create a SentenceTransformer model with mean pooling
model = SentenceTransformer("bert-base-uncased")
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens)
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
WARNING:sentence_transformers.SentenceTransformer:No sentence-transformers model found with name
```

```
# Evaluate the Base Embedding Model Using MTEB
```

```
# Import the MTEB class used to evaluate embedding models on benchmark tasks
from mteb import MTEB

# Create an MTEB evaluation object for the Banking77 classification task
evaluation = MTEB(tasks=["Banking77Classification"])

# Run the selected evaluation task using the embedding model
results = evaluation.run(model)
```

```
/usr/local/lib/python3.12/dist-packages/mteb/evaluation/MTEB.py:120: UserWarning: Passing task r
warnings.warn(
```

Selected tasks

Classification

- Banking77Classification, s2s

WARNING:mteb.abstasks.AbsTask:Dataset 'Banking77Classification' is superseded by 'Banking77Class

train.jsonl: 1.25M/? [00:00<00:00, 42.6MB/s]

test.jsonl: 365k/? [00:00<00:00, 25.6MB/s]

```
print(results)
```

```
[TaskResult(task_name=Banking77Classification, scores=...)]
```

```
# LOSS FUNCTIONS
```

```
# USING COSINE SIMILARITY
```

```
!pip install -U "datasets==3.1.0" "huggingface-hub==0.26.2"
```

```
Requirement already satisfied: datasets==3.1.0 in /usr/local/lib/python3.12/dist-packages (3.1.0)
Requirement already satisfied: huggingface-hub==0.26.2 in /usr/local/lib/python3.12/dist-packages (0.26.2)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from datasets==3.1.0) (3.12.2)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from datasets==3.1.0) (1.24.2)
Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.12/dist-packages (from datasets==3.1.0) (15.0.0)
Requirement already satisfied: dill<0.3.9,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from datasets==3.1.0) (0.3.8)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from datasets==3.1.0) (2.1.4)
Requirement already satisfied: requests>=2.32.2 in /usr/local/lib/python3.12/dist-packages (from datasets==3.1.0) (2.32.2)
Requirement already satisfied: tqdm>=4.66.3 in /usr/local/lib/python3.12/dist-packages (from datasets==3.1.0) (4.66.3)
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from datasets==3.1.0) (3.5.0)
Requirement already satisfied: multiprocess<0.70.17 in /usr/local/lib/python3.12/dist-packages (from datasets==3.1.0) (0.70.17)
Requirement already satisfied: fsspec<=2024.9.0,>=2023.1.0 in /usr/local/lib/python3.12/dist-packages (from datasets==3.1.0) (2024.9.0)
Requirement already satisfied: aiohttp in /usr/local/lib/python3.12/dist-packages (from datasets==3.1.0) (3.9.1)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from datasets==3.1.0) (24.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from datasets==3.1.0) (6.0.1)
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.12/dist-packages (from datasets==3.1.0) (4.9.0)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from datasets==3.1.0) (2.5.0)
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from datasets==3.1.0) (1.4.0)
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from datasets==3.1.0) (25.1.0)
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from datasets==3.1.0) (1.4.1)
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from datasets==3.1.0) (6.0.0)
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from datasets==3.1.0) (0.2.0)
Requirement already satisfied: yarll<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from datasets==3.1.0) (1.17.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from datasets==3.1.0) (3.3.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from datasets==3.1.0) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from datasets==3.1.0) (2.2.3)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from datasets==3.1.0) (2024.7.4)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from datasets==3.1.0) (2.8.2)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from datasets==3.1.0) (2024.1)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from datasets==3.1.0) (2024.1)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2) (1.17.0)
```

```
# Prepare the Training Dataset for Cosine Similarity Loss
```

```
# Import the Dataset class and the function to load datasets
from datasets import Dataset, load_dataset

# Load the first 50,000 training examples from the MNLI dataset
# Labels: 0 = Entailment, 1 = Neutral, 2 = Contradiction
train_dataset = load_dataset(
    "glue", "mnli", split="train"
).select(range(50_000))

# Remove the unnecessary index column
train_dataset = train_dataset.remove_columns("idx")

# Convert the original labels into similarity scores
# Entailment 0 = 1 Similar
# Neutral 1 = 0 Not Similar
# Contradiction 2 = 0 Not Similar
mapping = {2: 0, 1: 0, 0: 1}

# Create a new dataset with sentence pairs and similarity labels
train_dataset = Dataset.from_dict({

    # Store the premise as the first sentence
    "sentence1": train_dataset["premise"],

    # Store the hypothesis as the second sentence
    "sentence2": train_dataset["hypothesis"],

    # Convert each label to a float similarity score (0.0 or 1.0)
    "label": [float(mapping[label]) for label in train_dataset["label"]]
})
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens)
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets
warnings.warn(
```

```
# Uninstall the existing versions of the required libraries
!pip uninstall -y sentence-transformers transformers huggingface-hub datasets
```

```
Found existing installation: sentence-transformers 3.4.1
Uninstalling sentence-transformers-3.4.1:
  Successfully uninstalled sentence-transformers-3.4.1
Found existing installation: transformers 4.46.3
Uninstalling transformers-4.46.3:
  Successfully uninstalled transformers-4.46.3
Found existing installation: huggingface-hub 0.26.2
Uninstalling huggingface-hub-0.26.2:
  Successfully uninstalled huggingface-hub-0.26.2
Found existing installation: datasets 3.1.0
Uninstalling datasets-3.1.0:
  Successfully uninstalled datasets-3.1.0
```

```
# Create the STSB Evaluator
```

```
# Import the evaluator used to evaluate the embedding model
from sentence_transformers.evaluation import EmbeddingSimilarityEvaluator

# Load the validation split of the STSB (Semantic Textual Similarity Benchmark) dataset
val_sts = load_dataset("glue", "stsb", split="validation")

# Create an evaluator to measure how well the embedding model captures sentence similarity
evaluator = EmbeddingSimilarityEvaluator(
```

```

# First sentence in each sentence pair
sentences1=val_sts["sentence1"],

# Second sentence in each sentence pair
sentences2=val_sts["sentence2"],

# Normalize the similarity scores from the range 0-5 to 0-1
scores=[score / 5 for score in val_sts["label"]],

# Compare the sentence embeddings using cosine similarity
main_similarity="cosine"
)

```

train-00000-of-	502k/502k [00:00<00:00, 1.84MB/s]
00001.parquet: 100%	
validation-00000-of-	151k/151k [00:00<00:00, 398kB/s]
00001.parquet: 100%	
test-00000-of-00001.parquet: 100%	114k/114k [00:00<00:00, 267kB/s]
Generating train split: 100%	5749/5749 [00:00<00:00, 182953.24 examples/s]

Train the Embedding Model Using Cosine Similarity Loss

```

# Import the SentenceTransformer model, loss functions, trainer, and training arguments
from sentence_transformers import losses, SentenceTransformer
from sentence_transformers.trainer import SentenceTransformerTrainer
from sentence_transformers.training_args import SentenceTransformerTrainingArguments

# Load the base BERT model and create a SentenceTransformer model
embedding_model = SentenceTransformer("bert-base-uncased")

# Define the Cosine Similarity Loss used to train the embedding model
train_loss = losses.CosineSimilarityLoss(model=embedding_model)

# Define the training configuration
args = SentenceTransformerTrainingArguments(

    # Folder where the trained model and checkpoints will be saved
    output_dir="cosineloss_embedding_model",

    # Train the model for one complete pass through the dataset
    num_train_epochs=1,

    # Number of training examples processed in one batch on each device
    per_device_train_batch_size=32,

    # Number of evaluation examples processed in one batch on each device
    per_device_eval_batch_size=32,

    # Number of warmup steps before reaching the full learning rate
    warmup_steps=100,

    # Enable mixed precision (FP16) training to reduce memory usage and speed up training
    fp16=True,

    # Evaluate the model after every 100 training steps
    eval_steps=100,

    # Display the training loss after every 100 training steps
    logging_steps=100,
)

```

```
)

# Create the trainer by providing all the required components
trainer = SentenceTransformerTrainer(

    # The embedding model to be trained
    model=embedding_model,

    # The training configuration
    args=args,

    # The dataset used for training
    train_dataset=train_dataset,

    # The loss function used during training
    loss=train_loss,

    # The evaluator used to measure the model's performance
    evaluator=evaluator
)

# Start training the embedding model
trainer.train()
```

WARNING:sentence_transformers.SentenceTransformer:No sentence-transformers model found with name
 config.json: 100% 570/570 [00:00<00:00, 44.4kB/s]
 model.safetensors: 100% 440M/440M [00:04<00:00, 140MB/s]
 tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 4.73kB/s]
 vocab.txt: 232k/? [00:00<00:00, 2.00MB/s]
 tokenizer.json: 466k/? [00:00<00:00, 958kB/s]

wandb: WARNING The `run\_name` is currently set to the same value as `TrainingArguments.output\_dir`
 /usr/local/lib/python3.12/dist-packages/notebook/notebookapp.py:191: SyntaxWarning: invalid escape
 | | | | ' _ \ / _ ' / _ ' | _ / - _)

wandb: (1) Create a W&B account

wandb: (2) Use an existing W&B account

wandb: (3) Don't visualize my results

wandb: Enter your choice: 3

wandb: You chose "Don't visualize my results"

wandb: Using W&B in offline mode.

wandb: W&B API key is configured. Use `wandb login --relogin` to force relogin

Tracking run with wandb version 0.28.0

W&B syncing is set to `offline` in this directory. Run `wandb online` or set WANDB_MODE=online to
 enable cloud syncing.

Run data is saved locally in /content/wandb/offline-run-20260728_151226-eh8u96ss

View this run in the terminal with `wandb leet`

wandb: Detected [huggingface_hub.inference] in use.

wandb: Use W&B Weave for improved LLM call tracing. Install Weave with `pip install weave` then

wandb: For more information, check out the docs at: <https://weave-docs.wandb.ai>

[1563/1563 06:55, Epoch 1/1]

Step Training Loss

100	0.229500
200	0.171700
300	0.170800
400	0.157600
500	0.152600
600	0.156200
700	0.149100
800	0.156200
900	0.149900
1000	0.147300
1100	0.148500
1200	0.146800
1300	0.146300
1400	0.139100
1500	0.140000

TrainOutput(global_step=1563, training_loss=0.1568834221828312, metrics={'train_runtime':
 435.5923, 'train_samples_per_second': 114.786, 'train_steps_per_second': 3.588, 'total_flos':
 0.0, 'train_loss': 0.1568834221828312, 'epoch': 1.0})

Evaluate our trained model
 evaluator(embedding_model)

{'pearson_cosine': np.float64(0.7225834798707286),
 'spearman_cosine': np.float64(0.7253315788508788)}

```
# MULTIPLE NEGATIVES RANKING LOSS
```

```
# Prepare the Training Dataset for Multiple Negatives Ranking Loss
```

```
# Import the random module for shuffling, tqdm for progress bar, and Dataset utilities
import random
from tqdm import tqdm
from datasets import Dataset, load_dataset

# Load the first 50,000 training examples from the MNLI dataset
mnli = load_dataset("glue", "mnli", split="train").select(range(50_000))

# Remove the unnecessary index column
mnli = mnli.remove_columns("idx")

# Keep only the sentence pairs labeled as Entailment (label = 0)
mnli = mnli.filter(lambda x: True if x["label"] == 0 else False)

# Create a dictionary to store the training triplets
# Anchor -> Premise sentence
# Positive -> Correct hypothesis
# Negative -> Incorrect (random) hypothesis
train_dataset = {
    "anchor": [],
    "positive": [],
    "negative": []
}

# Copy all hypothesis sentences to create soft negatives
soft_negatives = mnli["hypothesis"]

# Randomly shuffle the hypothesis sentences
# This creates incorrect (soft negative) sentence pairs
random.shuffle(soft_negatives)

# Create the training triplets
for row, soft_negative in tqdm(zip(mnli, soft_negatives)):

    # Add the premise as the anchor sentence
    train_dataset["anchor"].append(row["premise"])

    # Add the correct hypothesis as the positive sentence
    train_dataset["positive"].append(row["hypothesis"])

    # Add the randomly shuffled hypothesis as the negative sentence
    train_dataset["negative"].append(soft_negative)

# Convert the dictionary into a Hugging Face Dataset
train_dataset = Dataset.from_dict(train_dataset)
```

```
16875it [00:01, 9366.80it/s]
```

```
# Create the STSB Evaluator
```

```
# Import the evaluator used to evaluate the embedding model
from sentence_transformers.evaluation import EmbeddingSimilarityEvaluator

# Load the validation split of the STSB (Semantic Textual Similarity Benchmark) dataset
val_sts = load_dataset("glue", "stsb", split="validation")
```

```
# Create an evaluator to measure the semantic similarity performance of the embedding model
evaluator = EmbeddingSimilarityEvaluator(

    # First sentence in each sentence pair
    sentences1=val_sts["sentence1"],

    # Second sentence in each sentence pair
    sentences2=val_sts["sentence2"],

    # Normalize the similarity scores from the range 0-5 to 0-1
    scores=[score / 5 for score in val_sts["label"]],

    # Use cosine similarity to compare the sentence embeddings
    main_similarity="cosine"
)
```

The cache for model files in Transformers v4.22.0 has been updated. Migrating your old cache. Th
0/0 [00:00<?, ?it/s]

train-00000-of- 502k/502k [00:00<00:00, 2.09MB/s]

00001.parquet: 100%

validation-00000-of- 151k/151k [00:00<00:00, 375kB/s]

00001.parquet: 100%

test-00000-of-00001.parquet: 100% 114k/114k [00:00<00:00, 280kB/s]

Generating train split: 100% 5749/5749 [00:00<00:00, 147744.31 examples/s]

Generating validation split: 100% 1500/1500 [00:00<00:00, 69403.05 examples/s]

Train the Embedding Model Using Multiple Negatives Ranking Loss

```
from sentence_transformers import losses, SentenceTransformer
from sentence_transformers.trainer import SentenceTransformerTrainer
from sentence_transformers.training_args import SentenceTransformerTrainingArguments

# Define model
embedding_model = SentenceTransformer('bert-base-uncased')

# Loss function
train_loss = losses.MultipleNegativesRankingLoss(model=embedding_model)

# Define the training arguments
args = SentenceTransformerTrainingArguments(
    output_dir="mnrls_embedding_model",
    num_train_epochs=1,
    per_device_train_batch_size=32,
    per_device_eval_batch_size=32,
    warmup_steps=100,
    fp16=
    True,
    eval_steps=100,
    logging_steps=100,
)

# Train model
trainer = SentenceTransformerTrainer(
    model=embedding_model,
    args=args,
    train_dataset=train_dataset,
    loss=train_loss,
    evaluator=evaluator
)
```



```
trainer.train()
```

WARNING:sentence_transformers.SentenceTransformer:No sentence-transformers model found with name
config.json: 100% 570/570 [00:00<00:00, 42.4kB/s]

model.safetensors: 100% 440M/440M [00:04<00:00, 130MB/s]

tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 4.19kB/s]

vocab.txt: 232k/? [00:00<00:00, 7.36MB/s]

tokenizer.json: 466k/? [00:00<00:00, 9.46MB/s]

wandb: WARNING The `run\_name` is currently set to the same value as `TrainingArguments.output\_dir`
/usr/local/lib/python3.12/dist-packages/notebook/notebookapp.py:191: SyntaxWarning: invalid escape
|_| | '_ \ / _ ' / _ ' | _/ -_)

wandb: (1) Create a W&B account

wandb: (2) Use an existing W&B account

wandb: (3) Don't visualize my results

wandb: Enter your choice: 3

wandb: You chose "Don't visualize my results"

wandb: Using W&B in offline mode.

wandb: W&B API key is configured. Use `wandb login --relogin` to force relogin

Tracking run with wandb version 0.28.0

W&B syncing is set to `offline` in this directory. Run `wandb online` or set WANDB_MODE=online to enable cloud syncing.

Run data is saved locally in /content/wandb/offline-run-20260729_050715-plogqa1w

View this run in the terminal with `wandb leet`

wandb: Detected [huggingface_hub.inference] in use.

wandb: Use W&B Weave for improved LLM call tracing. Install Weave with `pip install weave` then

wandb: For more information, check out the docs at: <https://weave-docs.wandb.ai>

[528/528 02:23, Epoch 1/1]

Step	Training Loss
100	0.342900
200	0.099000
300	0.078300
400	0.070300
500	0.071900

TrainOutput(global_step=528, training_loss=0.12882633100856433, metrics={'train_runtime': 204.0407, 'train_samples_per_second': 82.704, 'train_steps_per_second': 2.588, 'total_flos': 0.0, 'train_loss': 0.12882633100856433, 'epoch': 1.0})

```
# Evaluate our trained model
```

```
evaluator(embedding_model)
```

```
{'pearson_cosine': np.float64(0.8078362349076061),  
'spearman_cosine': np.float64(0.8123032027602317)}
```

Fine-Tuning an Embedding Model

Supervised

```
!pip uninstall -y datasets huggingface_hub  
!pip install "datasets==2.18.0" "huggingface_hub==0.23.5"
```

Found existing installation: datasets 2.18.0
Uninstalling datasets-2.18.0:
Successfully uninstalled datasets-2.18.0
Found existing installation: huggingface-hub 0.23.5

```

Uninstalling huggingface-hub-0.23.5:
  Successfully uninstalled huggingface-hub-0.23.5
Collecting datasets==2.18.0
  Using cached datasets-2.18.0-py3-none-any.whl.metadata (20 kB)
Collecting huggingface_hub==0.23.5
  Using cached huggingface_hub-0.23.5-py3-none-any.whl.metadata (12 kB)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from datasets)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from datasets)
Requirement already satisfied: pyarrow>=12.0.0 in /usr/local/lib/python3.12/dist-packages (from datasets)
Requirement already satisfied: pyarrow-hotfix in /usr/local/lib/python3.12/dist-packages (from datasets)
Requirement already satisfied: dill<0.3.9,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from datasets)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from datasets)
Requirement already satisfied: requests>=2.19.0 in /usr/local/lib/python3.12/dist-packages (from datasets)
Requirement already satisfied: tqdm>=4.62.1 in /usr/local/lib/python3.12/dist-packages (from datasets)
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from datasets)
Requirement already satisfied: multiprocessing in /usr/local/lib/python3.12/dist-packages (from datasets)
Requirement already satisfied: fsspec<=2024.2.0,>=2023.1.0 in /usr/local/lib/python3.12/dist-packages (from datasets)
Requirement already satisfied: aiohttp in /usr/local/lib/python3.12/dist-packages (from datasets)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from datasets)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from datasets)
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.12/dist-packages (from datasets)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from datasets)
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from datasets)
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from datasets)
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from datasets)
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from datasets)
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from datasets)
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from datasets)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from datasets)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from datasets)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from datasets)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from datasets)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from datasets)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from datasets)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from datasets)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from datasets)
Using cached datasets-2.18.0-py3-none-any.whl (510 kB)
Using cached huggingface_hub-0.23.5-py3-none-any.whl (402 kB)
Installing collected packages: huggingface_hub, datasets
ERROR: pip's dependency resolver does not currently take into account all the packages that are
diffusers 0.39.0 requires huggingface-hub<2.0,>=0.34.0, but you have huggingface-hub 0.23.5 which
transformers 5.13.1 requires huggingface-hub<2.0,>=1.5.0, but you have huggingface-hub 0.23.5 which
peft 0.19.1 requires huggingface-hub>=0.25.0, but you have huggingface-hub 0.23.5 which is incom
gradio 6.20.0 requires huggingface-hub<2.0,>=1.2.0, but you have huggingface-hub 0.23.5 which is
Successfully installed datasets-2.18.0 huggingface_hub-0.23.5

```

```
# Load the Training and Evaluation Datasets
```

```

# Import the function to load datasets and the evaluator for embedding models
from datasets import load_dataset
from sentence_transformers.evaluation import EmbeddingSimilarityEvaluator

# Load the first 50,000 training examples from the MNLI dataset
# Labels: 0 = Entailment, 1 = Neutral, 2 = Contradiction
train_dataset = load_dataset(
    "glue", "mnli", split="train"
).select(range(50_000))

# Remove the unnecessary index column
train_dataset = train_dataset.remove_columns("idx")

# Load the validation split of the STSB (Semantic Textual Similarity Benchmark) dataset
val_sts = load_dataset("glue", "sts_b", split="validation")

# Create an evaluator to measure the semantic similarity performance of the embedding model
evaluator = EmbeddingSimilarityEvaluator(

    # First sentence in each sentence pair

```

```

sentences1=val_sts["sentence1"],

# Second sentence in each sentence pair
sentences2=val_sts["sentence2"],

# Normalize the similarity scores from the range 0-5 to 0-1
scores=[score / 5 for score in val_sts["label"]],

# Use cosine similarity to compare the sentence embeddings
main_similarity="cosine"
)

```

The cache for model files in Transformers v4.22.0 has been updated. Migrating your old cache. Th
 0/0 [00:00<?, ?it/s]

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
 The secret `HF\_TOKEN` does not exist in your Colab secrets.
 To authenticate with the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings/tokens>).
 You will be able to reuse this secret in all of your notebooks.
 Please note that authentication is recommended but still optional to access public models or datasets.

warnings.warn(

train-00000-of-100000.parquet: 100% 52.2M/52.2M [00:04<00:00, 9.09MB/s]

00001.parquet: 100%

(...)validation_matched-00000-of-100000.parquet: 100% 1.21M/1.21M [00:00<00:00, 4.85MB/s]

00001.parquet: 100%

(...)validation_mismatched-00000-of-100000.parquet: 100% 1.25M/1.25M [00:00<00:00, 75.8MB/s]

00001.parquet: 100%

test_matched-00000-of-100000.parquet: 100% 1.22M/1.22M [00:00<00:00, 5.50MB/s]

00001.parquet: 100%

test_mismatched-00000-of-100000.parquet: 100% 1.26M/1.26M [00:00<00:00, 2.78MB/s]

00001.parquet: 100%

Generating train split: 100% 392702/392702 [00:00<00:00, 676297.00 examples/s]

Generating validation_matched split: 100% 9815/9815 [00:00<00:00, 202640.84 examples/s]

Generating validation_mismatched split: 100% 9832/9832 [00:00<00:00, 219774.02 examples/s]

Generating test_matched split: 100% 9796/9796 [00:00<00:00, 253417.89 examples/s]

Generating test_mismatched split: 100% 9847/9847 [00:00<00:00, 261192.41 examples/s]

train-00000-of-100000.parquet: 100% 502k/502k [00:00<00:00, 1.99MB/s]

00001.parquet: 100%

validation-00000-of-100000.parquet: 100% 151k/151k [00:00<00:00, 12.7MB/s]

00001.parquet: 100%

test-00000-of-00001.parquet: 100% 114k/114k [00:00<00:00, 712kB/s]

Generating train split: 100% 5749/5749 [00:00<00:00, 182286.60 examples/s]

```
# Model and loss function
```

```

from sentence_transformers import losses, SentenceTransformer
from sentence_transformers.trainer import SentenceTransformerTrainer
from sentence_transformers.training_args import SentenceTransformerTrainingArguments

# Define model
embedding_model = SentenceTransformer('sentence-transformers/all-MiniLM-L6-v2')

```

```
# Loss function
train_loss = losses.MultipleNegativesRankingLoss(model=embedding_model)
```

```
modules.json: 100% 349/349 [00:00<00:00, 31.4kB/s]
config_sentence_transformers.json: 100% 116/116 [00:00<00:00, 13.3kB/s]
README.md: 10.5k/? [00:00<00:00, 1.19MB/s]
sentence_bert_config.json: 100% 53.0/53.0 [00:00<00:00, 3.70kB/s]
config.json: 100% 612/612 [00:00<00:00, 34.9kB/s]
model.safetensors: 100% 90.9M/90.9M [00:00<00:00, 113MB/s]
tokenizer_config.json: 100% 350/350 [00:00<00:00, 21.1kB/s]
vocab.txt: 232k/? [00:00<00:00, 10.8MB/s]
tokenizer.json: 466k/? [00:00<00:00, 18.2MB/s]
special_tokens_map.json: 100% 112/112 [00:00<00:00, 13.6kB/s]
config.json: 100% 190/190 [00:00<00:00, 21.4kB/s]
```

Configure the Fine-Tuning Settings

```
# Define the training arguments
args = SentenceTransformerTrainingArguments(
    output_dir="finetuned_embedding_model",
    num_train_epochs=1,
    per_device_train_batch_size=32,
    per_device_eval_batch_size=32,
    warmup_steps=100,
    fp16=
    True
    ,
    eval_steps=100,
    logging_steps=100,
)
```

Fine-Tune the Pretrained Embedding Model

```
# Train model
trainer = SentenceTransformerTrainer(
    model=embedding_model,
    args=args,
    train_dataset=train_dataset,
    loss=train_loss,
    evaluator=evaluator
)
trainer.train()
```

```
wandb: WARNING The `run_name` is currently set to the same value as `TrainingArguments.output_dir`
/usr/local/lib/python3.12/dist-packages/notebook/notebookapp.py:191: SyntaxWarning: invalid escape
|_| | '_ \ / _ ' | _/ -_)
wandb: (1) Create a W&B account
wandb: (2) Use an existing W&B account
wandb: (3) Don't visualize my results
wandb: Enter your choice: 3
wandb: You chose "Don't visualize my results"
wandb: Using W&B in offline mode.
wandb: W&B API key is configured. Use `wandb login --relogin` to force relogin
Tracking run with wandb version 0.28.0
W&B syncing is set to `offline` in this directory. Run `wandb online` or set WANDB_MODE=online to
enable cloud syncing.
Run data is saved locally in /content/wandb/offline-run-20260802_163918-oiev9ag6
View this run in the terminal with `wandb leet`
wandb: Detected [huggingface_hub.inference] in use.
wandb: Use W&B Weave for improved LLM call tracing. Install Weave with `pip install weave` then
wandb: For more information, check out the docs at: https://weave-docs.wandb.ai
WARNING:sentence_transformers.data_collator:Column 'hypothesis' is at index 1, whereas a column
dataset = dataset.select_columns(['hypothesis', 'entailment', 'contradiction'])
[1563/1563 01:43, Epoch 1/1]
```

Step Training Loss

100	0.158300
200	0.113100
300	0.122400
400	0.119600
500	0.110400
600	0.101700
700	0.121300
800	0.101700
900	0.102500
1000	0.104300
1100	0.093100
1200	0.107000
1300	0.099600
1400	0.106700
1500	0.104500

```
TrainOutput(global_step=1563, training_loss=0.11036009462079556, metrics={'train_runtime':
167.7037, 'train_samples_per_second': 298.145, 'train_steps_per_second': 9.32, 'total_flos':
0.0, 'train_loss': 0.11036009462079556, 'epoch': 1.0})
```

```
# Evaluate our trained model
evaluator(embedding_model)
```

```
{'pearson_cosine': np.float64(0.8492933259217872),
'spearman_cosine': np.float64(0.8491207745994088)}
```

Augmented SBERT

```
# Prepare the Gold Dataset
```

```

# Import the Pandas library for handling tabular data
import pandas as pd

# Import tqdm to display a progress bar
from tqdm import tqdm

# Import the Dataset class and the function to load datasets
from datasets import load_dataset, Dataset

# Import the InputExample class used to create training examples
from sentence_transformers import InputExample

# Import the dataloader that avoids duplicate sentence pairs in a batch
from sentence_transformers.datasets import NoDuplicatesDataLoader

# Load the first 10,000 training examples from the MNLI dataset
dataset = load_dataset(
    "glue", "mnli", split="train"
).select(range(10_000))

# Map the original labels to similarity labels
mapping = {2: 0, 1: 0, 0: 1}

# Create InputExample objects for training the Cross-Encoder
gold_examples = [
    InputExample(

        # Store the premise and hypothesis as a sentence pair
        texts=[row["premise"], row["hypothesis"]],

        # Assign the mapped label to the sentence pair
        label=mapping[row["label"]]
    )

    # Repeat for every row in the dataset and show the progress
    for row in tqdm(dataset)
]

# Create a dataloader with a batch size of 32
gold_dataloader = NoDuplicatesDataLoader(
    gold_examples,
    batch_size=32
)

# Create a Pandas DataFrame from the dataset
gold = pd.DataFrame({

    # Store the premise as the first sentence
    "sentence1": dataset["premise"],

    # Store the hypothesis as the second sentence
    "sentence2": dataset["hypothesis"],

    # Store the mapped labels
    "label": [mapping[label] for label in dataset["label"]]
})

```

100%|██████████| 10000/10000 [00:00<00:00, 20654.96it/s]

```

# STEP 1
# Train the Cross-Encoder

```

```

from sentence_transformers.cross_encoder import CrossEncoder
# Train a cross-encoder on the gold dataset

```

```

cross_encoder = CrossEncoder("bert-base-uncased", num_labels=2)
cross_encoder.fit(
train_dataloader=gold_dataloader,
epochs=1,
show_progress_bar=True,
warmup_steps=100,
use_amp=False
)

```

Some weights of BertForSequenceClassification were not initialized from the model checkpoint at You should probably TRAIN this model on a down-stream task to be able to use it for predictions

Epoch: 100%

1/1 [02:39<00:00, 159.18s/it]

Iteration: 100%

312/312 [02:39<00:00, 2.00it/s]

```

# STEP 2
# Prepare the Silver Dataset

```

```

# Load the remaining 40,000 MNLI sentence pairs as the unlabeled silver dataset
silver = load_dataset(
    "glue", "mnli", split="train"
).select(range(10_000, 50_000))

# Create a list of (premise, hypothesis) sentence pairs for prediction
pairs = list(zip(silver["premise"], silver["hypothesis"]))

```

```

# STEP 3
# Generate Labels for the Silver Dataset

```

```

# Import the NumPy library for numerical operations
import numpy as np

# Predict labels for the silver sentence pairs using the fine-tuned Cross-Encoder
output = cross_encoder.predict(
    pairs,
    apply_softmax=True,
    show_progress_bar=True
)

# Create a Pandas DataFrame with the predicted labels
silver = pd.DataFrame({

    # Store the premise as the first sentence
    "sentence1": silver["premise"],

    # Store the hypothesis as the second sentence
    "sentence2": silver["hypothesis"],

    # Assign the predicted label with the highest probability
    "label": np.argmax(output, axis=1)
})

```

Batches: 100%

1250/1250 [03:50<00:00, 6.24it/s]

```

# Create the Augmented Training Dataset and STSB Evaluator

```

```

# Combine the gold and silver datasets into a single dataset
data = pd.concat([gold, silver], ignore_index=True, axis=0)

```

```

# Remove duplicate sentence pairs from the combined dataset
data = data.drop_duplicates(
    subset=["sentence1", "sentence2"],
    keep="first"
)

# Convert the Pandas DataFrame into a Hugging Face Dataset
train_dataset = Dataset.from_pandas(
    data,
    preserve_index=False
)

# Import the evaluator used to evaluate the embedding model
from sentence_transformers.evaluation import EmbeddingSimilarityEvaluator

# Load the validation split of the STSB dataset
val_sts = load_dataset("glue", "stsb", split="validation")

# Create an evaluator for measuring semantic similarity
evaluator = EmbeddingSimilarityEvaluator(

    # First sentence in each sentence pair
    sentences1=val_sts["sentence1"],

    # Second sentence in each sentence pair
    sentences2=val_sts["sentence2"],

    # Normalize the similarity scores from 0-5 to 0-1
    scores=[score / 5 for score in val_sts["label"]],

    # Compare embeddings using cosine similarity
    main_similarity="cosine"
)

```

```
# Train the Augmented SBERT Model
```

```

from sentence_transformers import losses, SentenceTransformer
from sentence_transformers.trainer import SentenceTransformerTrainer
from sentence_transformers.training_args import SentenceTransformerTrainingArguments

# Define model
embedding_model = SentenceTransformer("bert-base-uncased")

# Loss function
train_loss = losses.CosineSimilarityLoss(model=embedding_model)

# Define the training arguments
args = SentenceTransformerTrainingArguments(
    output_dir="augmented_embedding_model",
    num_train_epochs=1,
    per_device_train_batch_size=32,
    per_device_eval_batch_size=32,
    warmup_steps=100,
    fp16=True,
    eval_steps=100,
    logging_steps=100,
)

# Train model
trainer = SentenceTransformerTrainer(
    model=embedding_model,
    args=args,
    train_dataset=train_dataset,

```



```
loss=train_loss,
evaluator=evaluator
)
trainer.train()
```

WARNING:sentence_transformers.SentenceTransformer:No sentence-transformers model found with name

 [943/1563 03:23 < 02:14, 4.62 it/s, Epoch 0.60/1]

Step	Training Loss
------	---------------

100	0.225600
200	0.164100
300	0.151600
400	0.146800
500	0.145200
600	0.141500
700	0.137600
800	0.133500
900	0.136900

 [1563/1563 05:52, Epoch 1/1]

Step	Training Loss
------	---------------

100	0.225600
200	0.164100
300	0.151600
400	0.146800
500	0.145200
600	0.141500
700	0.137600
800	0.133500
900	0.136900
1000	0.133600
1100	0.133400
1200	0.134500
1300	0.131200
1400	0.132000
1500	0.135000

TrainOutput(global_step=1563, training_loss=0.14492476787310873, metrics={'train_runtime': 352.5805, 'train_samples_per_second': 141.806, 'train_steps_per_second': 4.433, 'total_flos': 0.0, 'train_loss': 0.14492476787310873, 'epoch': 1.0})

```
evaluator(embedding_model)
```

```
{'pearson_cosine': np.float64(0.701837925814763),
'spearman_cosine': np.float64(0.7071925218489362)}
```

Unsupervised Learning

Transformer-Based Sequential Denoising Auto-Encoder

Download the Required NLTK Tokenizers

```
# Import the NLTK library for natural language processing tasks
import nltk

# Download the 'punkt' tokenizer used to split text into sentences and words
nltk.download("punkt")

# If you're using a newer version of NLTK may also need punkt_tab:
# Download the additional Punkt tokenizer data required by newer NLTK versions
nltk.download("punkt_tab")
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt_tab.zip.
True
```

Prepare the TSDAE Training Dataset

```
# Import tqdm to display a progress bar during dataset creation
from tqdm import tqdm

# Import the Dataset class and the function to load datasets
from datasets import Dataset, load_dataset

# Import the dataset class used to create damaged and original sentence pairs for TSDAE training
from sentence_transformers.datasets import DenoisingAutoEncoderDataset

# Load the first 25,000 training examples from the MNLI dataset
mnli = load_dataset("glue", "mnli", split="train").select(range(25_000))

# Combine all premise and hypothesis sentences into a single list of sentences
flat_sentences = mnli["premise"] + mnli["hypothesis"]

# Remove duplicate sentences and automatically create damaged-original sentence pairs
damaged_data = DenoisingAutoEncoderDataset(list(set(flat_sentences)))

# Create a dictionary to store the damaged sentences and their corresponding original sentences:
train_dataset = {
    "damaged_sentence": [],
    "original_sentence": []
}

# Iterate through each damaged-original sentence pair and store them in the dictionary
for data in tqdm(damaged_data):

    # Store the damaged (noisy) version of the sentence
    train_dataset["damaged_sentence"].append(data.texts[0])

    # Store the corresponding original sentence
    train_dataset["original_sentence"].append(data.texts[1])

# Convert the dictionary into a Hugging Face Dataset for model training
train_dataset = Dataset.from_dict(train_dataset)
```

100%|██████████| 48353/48353 [00:11<00:00, 4323.23it/s]

```
train_dataset[0]
```

```
{'damaged_sentence': 'yeah yeah ours is point drives thinking know everybody here screaming about',  
 'original_sentence': "yeah yeah ours is seven point seven five almost eight and it drives me crazy i'm thinking you know everybody is sitting here screaming about"}
```

```
# Create the STSB Evaluator
```

```
from sentence_transformers.evaluation import EmbeddingSimilarityEvaluator  
  
# Create an embedding similarity evaluator for stsb  
val_sts = load_dataset("glue", "stsb", split="validation")  
evaluator = EmbeddingSimilarityEvaluator(  
    sentences1=val_sts["sentence1"],  
    sentences2=val_sts["sentence2"],  
    scores=[score/5 for score in val_sts["label"]],  
    main_similarity="cosine"  
)
```

```
# Build the TSDAE Embedding Model
```